MACHINE LEARNING MODEL ENGINEERING

Design and Evaluation of a Mini Production-Style Machine Learning System for Customer Churn Prediction

Prakash Babu M. | 2025EM1100094

Abstract

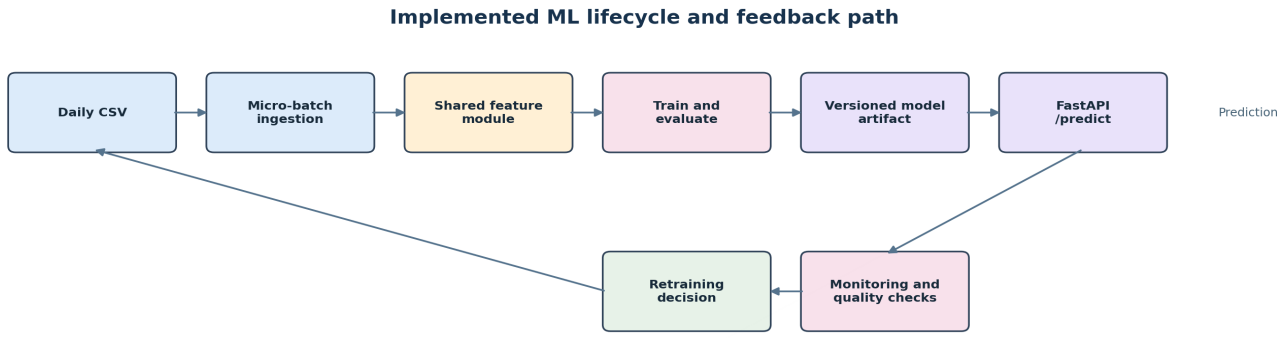
This project develops a production-style machine learning system for telecom customer churn prediction using 7,043 records from the IBM Telco Customer Churn dataset. A logistic-regression baseline and random-forest candidate were evaluated on the same stratified hold-out test set. The random forest achieved ROC-AUC 0.8457 and F1 0.6360 and was saved as a versioned model artifact. FastAPI provides online prediction, while supporting components implement micro-batch ingestion, automated verification, latency measurement, data-quality monitoring and retraining decisions. The design therefore evaluates the reproducibility and operation of the model, not only its predictive accuracy.

1. Introduction and problem definition

Customer churn reduces recurring revenue and may increase the cost of replacing lost customers. The analytical objective is to estimate the probability that an active telecom customer will leave, using account, service and billing attributes. A high score can prioritise a customer for retention review; it does not automatically determine pricing or deny service. A false positive may waste a retention offer, whereas a false negative may miss a customer who later leaves.

Project objectives

- Construct a repeatable data and feature-processing pipeline.
- Compare a simple baseline with a nonlinear candidate model.
- Package the selected pipeline for consistent online inference.
- Demonstrate ingestion, verification, monitoring and retraining logic.



2. Dataset, preprocessing and feature design

2.1 Dataset and target

The public IBM Telco Customer Churn sample contains demographic, account-tenure, subscribed-service, contract, payment and billing variables. The target variable, Churn, was mapped from Yes/No to 1/0. customerID was retained for ingestion deduplication but excluded from training because an arbitrary identifier should not influence the estimated risk. The observed positive-class rate was 26.54%, making accuracy insufficient as the only evaluation measure.

Dataset property	Observed value
Customer records	7,043
Raw input variables	19 model inputs plus customerID
Target	Churn (Yes = 1, No = 0)
Positive churn rate	26.54%
Training/test split	80% / 20%, stratified
Held-out test records	1,409

2.2 Cleaning and preprocessing

Raw records were cleaned before model fitting, while transformations that learn from data were fitted only on the training split. This prevents information from the held-out test set influencing the model. The applied transformations are summarised below.

Data issue or type	Transformation	Purpose
customerID	Excluded from model inputs	Avoid learning from an arbitrary identifier
Churn	Yes/No mapped to 1/0	Create the binary target
TotalCharges	Numeric conversion; 11 blanks replaced by MonthlyCharges x tenure	Use a deterministic inference-time rule
Numeric variables	Median imputation and standardisation	Handle missing values and comparable scales
Categorical variables	Most-frequent imputation and one-hot encoding	Create numeric model inputs
Unknown categories	Ignored by the fitted encoder	Prevent inference failure
Class imbalance	Balanced class weights	Reduce bias toward non-churn cases

2.3 Engineered features

Nine derived variables represent tenure, billing behaviour, contract flexibility and service engagement. They use information available for both historical and new customers.

Feature group	Derived variables	Purpose
Billing	avg_charge_per_month; charge_gap; estimated_ltv	Represent historical and current customer value
Tenure	tenure_years; is_new_customer	Represent account age and early-tenure risk
Contract and payment	is_month_to_month; has_auto_payment	Represent cancellation flexibility and payment friction
Service engagement	support_services_count; streaming_services_count	Count subscribed support and streaming products

2.4 Preventing training-serving skew

Training-serving skew occurs when records are processed differently during model development and production prediction. This project prevents it in two stages. One shared Python module performs cleaning and engineered-feature calculations for both training and the API. The fitted encoder, imputer, scaler and classifier are then saved together. For example, TotalCharges and tenure years are calculated identically for a historical row and a new API record.

Training: historical records -> shared features -> fitted preprocessing -> classifier -> saved artifact
Prediction: new record -> same features -> saved preprocessing -> saved classifier -> probability

3. Experimental methodology

3.1 Experimental design

The experiment asks whether a nonlinear candidate provides a useful improvement over a simpler baseline. The data was divided into 80% training and 20% testing using stratified sampling, which preserves the churn proportion. A fixed random seed of 42 makes the split repeatable. Learned preprocessing and model fitting used only the training portion; 1,409 test records were retained for comparison.

Load data -> clean and engineer features -> stratified split -> train baseline and candidate -> held-out evaluation -> promotion decision -> save versioned pipeline

3.2 Baseline and candidate models

A baseline is a credible, relatively simple model against which a candidate can be judged. Logistic regression provides a stable linear reference. Random forest was chosen as the candidate because multiple decision trees can represent nonlinear relationships and interactions among contract, tenure, charges and services. Balanced class weights were used because churn cases form 26.54% of the data.

Setting	Baseline	Candidate
Estimator	Logistic regression	Random forest
Class handling	Balanced	Balanced subsamples
Maximum iterations / trees	1,500 iterations	300 trees
Maximum depth	Not applicable	8
Minimum leaf size	Not applicable	6
Random seed	42	42

3.3 Evaluation measures

No single metric describes all churn errors. ROC-AUC measures how well the model ranks churners above non-churners across thresholds. Precision measures how many flagged customers actually churned; recall measures how many actual churners were found; and F1 balances the two. Accuracy and the confusion matrix were also reported at the configured probability threshold of 0.50.

Measure	Interpretation in this project
ROC-AUC	Quality of customer risk ranking across thresholds
Precision	Reliability of customers flagged for review
Recall	Coverage of customers who actually churned
F1	Balance between precision and recall

3.4 Promotion rule and reproducibility

Promotion means selecting the candidate as the version to serve. It was eligible when ROC-AUC was at least 0.80 and no more than 0.01 below the baseline. Storing this rule in JSON before final evaluation avoids an informal choice after seeing the results. The data checksum, random seed, dependencies, settings and version are also recorded so the run can be repeated.

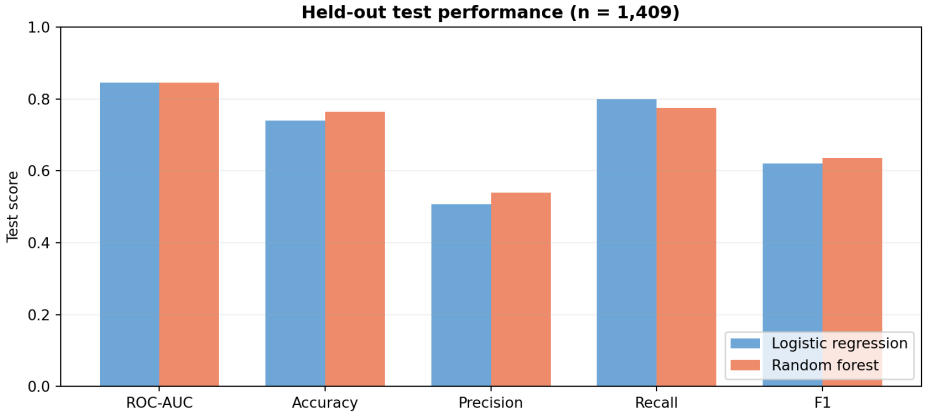
3.5 Artifact and verification strategy

The promoted Joblib artifact contains the fitted preprocessing, classifier, threshold, name and version. Automated tests verify feature calculations, API output, model health, ingestion deduplication and retraining triggers. JSON and text files retain training, benchmark and monitoring evidence.

4. Model results and API inference

4.1 Offline evaluation

Model	ROC-AUC	Accuracy	Precision	Recall	F1
Logistic regression	0.8456	0.7402	0.5068	0.7995	0.6203
Random forest	0.8457	0.7644	0.5390	0.7754	0.6360



4.2 Interpretation and selection

The models had nearly identical ranking performance: random forest improved ROC-AUC by only 0.0001. At threshold 0.50, it increased precision and F1 while recall decreased from 0.7995 to 0.7754. Thus, fewer non-churners were incorrectly flagged, but slightly more actual churners were missed. Its confusion matrix contained 787 true negatives, 248 false positives, 84 false negatives and 290 true positives. It satisfied the promotion rule and was saved as churn-rf-1.0.0, although its advantage was modest.

4.3 How a new API request is processed

FastAPI provides GET /health to confirm model availability and POST /predict to score one customer. When a request arrives, Pydantic first validates the required fields and types. The shared feature function then prepares the record, the saved preprocessing converts it to the model's numeric input, and predict_proba produces churn probability. The 0.50 threshold converts that probability to class 0 or 1. The response also returns model version for traceability.

Request -> validation -> shared feature engineering -> saved preprocessing -> random forest -> probability and model version

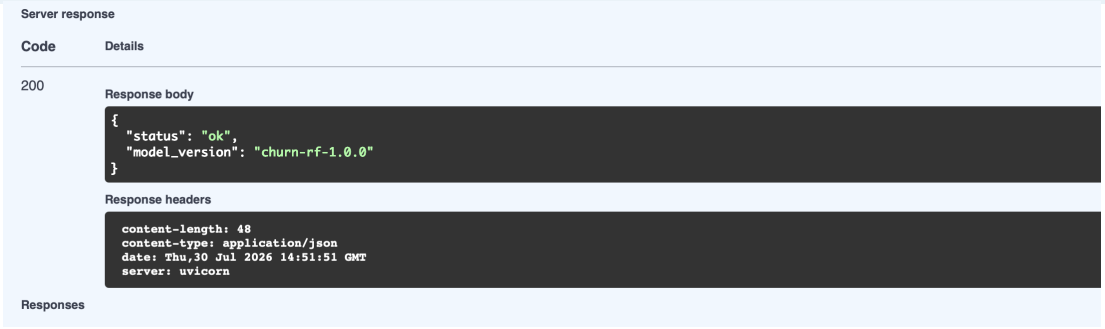


Figure 3. Swagger UI health response: HTTP 200, status ok and the loaded model version.

Figure 4. Swagger UI prediction response for a new customer: prediction 1, probability 0.758 and model version churn-rf-1.0.0.

5. Production engineering, monitoring and retraining

5.1 Ingestion and deployment pattern

A service agent may be waiting for a prediction during a customer interaction, so online request-response inference is appropriate. Customer account and billing features usually change over hours or days, not milliseconds; therefore, scheduled micro-batch ingestion is more suitable than continuous event streaming. The ingestion script merges each incoming CSV with the training table, retains the latest row for a repeated customerID and records row counts in an audit log. Batch scoring can support scheduled retention campaigns, while the API supports individual interactive decisions.

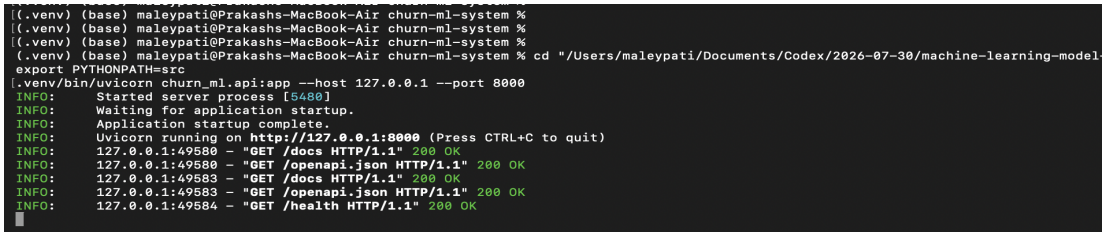


Figure 5. Terminal output showing Uvicorn startup and successful /docs and /health HTTP requests.

5.2 Performance measurement

Because a human may be waiting, the prototype sets a p95 model-service latency target below 100 ms, leaving additional time for network and user-interface processing. The latest TestClient benchmark issued 250 requests. Mean latency was 37.016 ms, p95 latency was 35.141 ms, and measured throughput was 27.02 requests per second. The measured p95 satisfies the prototype target. This sequential local test does not represent concurrent production traffic.

5.3 Monitoring design and implemented check

Monitoring separates service operation, input-data reliability and model usefulness. The proposed views, intended users and alerts are summarised below.

Monitoring view	Measures	Primary user	Alert condition
API operations	Health, p95 latency, HTTP errors	Operations engineer	Unavailable, errors increase, or p95 > 100 ms
Data quality and drift	Volume, missingness, schema and mean shifts	Data / ML engineer	Missingness > 5% or shift > 1 standard deviation
Model and business	Score distribution, delayed ROC-AUC, precision and recall	Model owner / retention manager	ROC-AUC drops > 0.03 below baseline

The implemented drift check compares MonthlyCharges, TotalCharges, tenure and avg_charge_per_month with training reference values. When a threshold is exceeded, it sets status to warning, stores descriptive messages in drift_report.json and prints each message through the command-line script. The latest run checked 7,043 rows and returned status ok with no warnings.

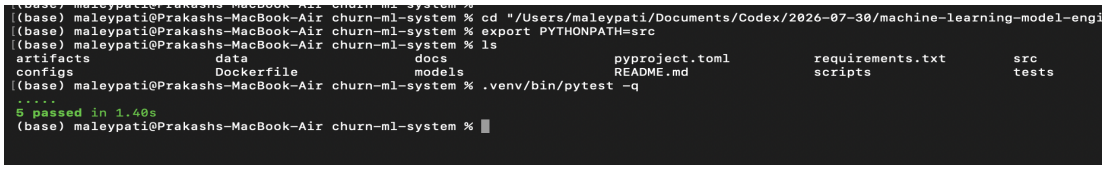


Figure 6. Terminal output showing that all five automated tests passed.

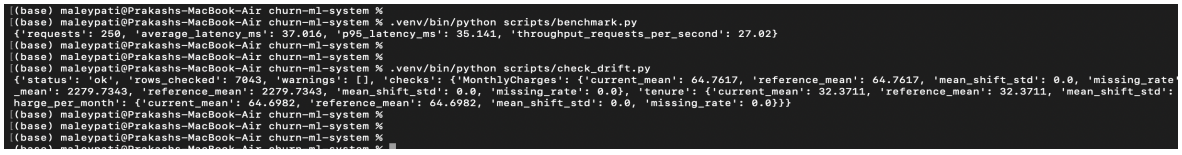


Figure 7. Terminal benchmark and drift-check outputs from the latest execution.

5.4 Retraining policy

Retraining is reconsidered after 30 days of new data, a recent ROC-AUC decrease greater than 0.03, or feature drift above one standard deviation. A trigger starts evaluation; it does not automatically replace the served model. New data must pass quality checks, and a challenger must satisfy the same promotion rule.

5.5 Incident scenario

Suppose an upstream system renames MonthlyCharges or sends currency strings instead of numbers. Validation or ingestion would fail. The appropriate response is to stop the affected ingestion, continue serving the last known-good model, correct the producer contract and add a regression test. Malformed data must not trigger retraining.

6. Discussion, limitations and conclusion

6.1 Engineering trade-offs

A production design must balance predictive performance, response time, interpretability and implementation complexity. The following trade-offs explain why the prototype uses relatively simple components.

Decision	Benefit	Limitation
Online API	Immediate score during an interaction	Requires low latency and service availability
Random forest	Models nonlinear relationships	Less interpretable than logistic regression
Shared feature module	Simple offline/online consistency	Needs stronger governance for many models
Mean-shift check	Low-cost and explainable	Does not detect every distribution change

6.2 Limitations and threats to validity

The dataset is static and has no timestamps, so a random split cannot show how performance changes under future market conditions. It also excludes complaints, network quality and previous retention offers. The reported metrics come from one hold-out split rather than repeated or temporal validation. The 0.50 threshold is reproducible but was not selected using the financial costs of false positives and negatives. The API was exercised through live local Uvicorn and Swagger UI, while the repeatable benchmark used FastAPI TestClient; neither represents concurrent cloud load. Monitoring produces JSON reports rather than a live dashboard and has no delayed churn labels. Therefore, this is a production-style prototype, not a validated commercial service.

6.3 Conclusion

The project implements a reproducible churn-model lifecycle: micro-batch ingestion, shared feature engineering, baseline-candidate evaluation, versioned packaging, online inference, automated testing, latency measurement, data-quality checks and retraining decisions. Random forest met the promotion criteria with ROC-AUC 0.8457 and F1 0.6360, although its improvement over logistic regression was modest. The principal result is therefore not only a churn score, but an integrated and verifiable process for training and operating the model.

GitHub repository

Source code and reproducibility artifacts: github.com/2025em1100094-alt/customer-churn-ml-system.

Reference

IBM. Telco Customer Churn sample dataset. github.com/IBM/telco-customer-churn-on-icp4d